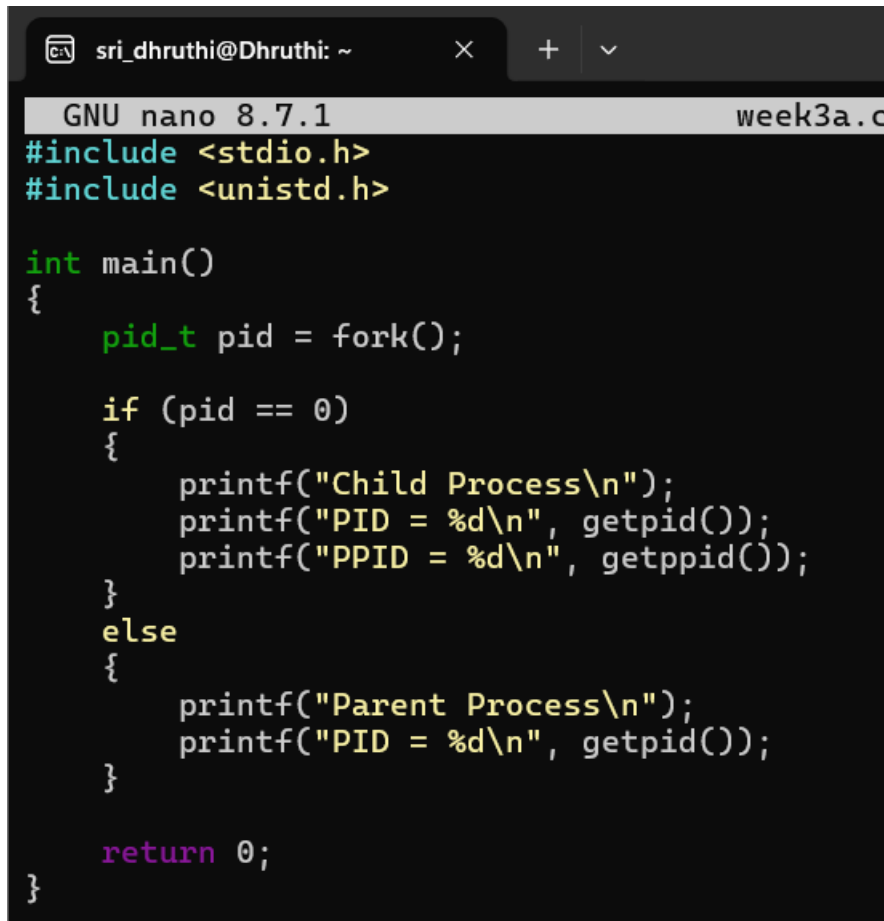


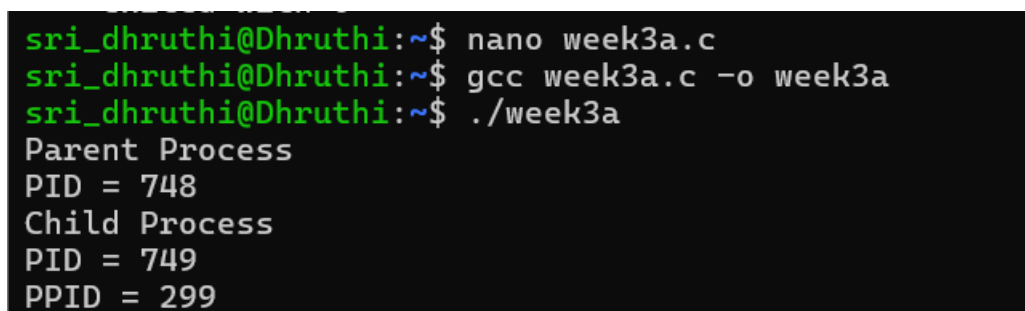
WEEK 3

Week 3A — Process Creation Using fork()

This program creates a child process using fork() and displays the process information. It demonstrates the relationship between the parent and child processes.



```
sri_dhruthi@Dhruthi: ~  
GNU nano 8.7.1 week3a.c  
#include <stdio.h>  
#include <unistd.h>  
  
int main()  
{  
    pid_t pid = fork();  
  
    if (pid == 0)  
    {  
        printf("Child Process\n");  
        printf("PID = %d\n", getpid());  
        printf("PPID = %d\n", getppid());  
    }  
    else  
    {  
        printf("Parent Process\n");  
        printf("PID = %d\n", getpid());  
    }  
  
    return 0;  
}
```



```
sri_dhruthi@Dhruthi:~$ nano week3a.c  
sri_dhruthi@Dhruthi:~$ gcc week3a.c -o week3a  
sri_dhruthi@Dhruthi:~$ ./week3a  
Parent Process  
PID = 748  
Child Process  
PID = 749  
PPID = 299
```

Week 3B — Process States and /proc

This program creates a child process and makes the parent wait for the child. You can observe the processes using ps, top, and /proc. This matches the process-state investigation required in Question 3.

```
sri_dhruthi@Dhruthi: ~  
GNU nano 8.7.1 week3b.c *  
#include <stdio.h>  
#include <unistd.h>  
#include <sys/wait.h>  
  
int main()  
{  
    pid_t pid = fork();  
  
    if (pid == 0)  
    {  
        printf("Child Process Running\n");  
        printf("Child PID = %d\n", getpid());  
        sleep(10);  
        printf("Child Process Finished\n");  
    }  
    else  
    {  
        printf("Parent Process\n");  
        printf("Parent PID = %d\n", getpid());  
  
        wait(NULL);  
  
        printf("Parent Process Continues\n");  
    }  
  
    return 0;  
}
```

```
sri_dhruthi@Dhruthi:~$ nano week3b.c  
sri_dhruthi@Dhruthi:~$ gcc week3b.c -o week3b  
sri_dhruthi@Dhruthi:~$ ./week3b  
Parent Process  
Parent PID = 806  
Child Process Running  
Child PID = 807  
Child Process Finished  
Parent Process Continues  
sri_dhruthi@Dhruthi:~$
```